네트워크 이론 정리 — DevOps 관점

> 작성일: 2026-04-01
> 대상: DevOps 입문~중급 학습자
> 범위: OSI/TCP-IP 모델부터 DNS, HTTP, 클라우드 네트워킹, 컨테이너 네트워킹, 서비스 메시까지

목차

1. 네트워크 기초 — 왜 DevOps에서 중요한가
2. OSI 7계층 모델
3. TCP/IP 모델
4. IP 주소와 서브넷
5. TCP와 UDP
6. 포트와 소켓
7. DNS (Domain Name System)
8. HTTP/HTTPS
9. 라우팅과 게이트웨이
10. 로드 밸런싱
11. 방화벽과 접근 제어
12. 클라우드 네트워킹 (VPC)
13. 컨테이너 네트워킹
14. Kubernetes 네트워킹
15. 서비스 메시 (Service Mesh)
16. 네트워크 트러블슈팅
17. 네트워크 보안 기초
18. 참고 자료

---

1. 네트워크 기초 — 왜 DevOps에서 중요한가

DevOps 인프라의 모든 것은 네트워크 위에서 동작한다.

- 컨테이너 간 통신 → 네트워크
- CI/CD 파이프라인의 아티팩트 전송 → 네트워크
- 마이크로서비스 API 호출 → 네트워크
- 클라우드 리소스 격리 (VPC, 서브넷) → 네트워크
- 로드 밸런서, CDN, DNS → 네트워크
- 프로덕션 장애의 상당수 → 네트워크 문제

네트워크를 모르면 클라우드 인프라 설계가 추측이 되고, 장애 대응이 불가능해진다.

---

2. OSI 7계층 모델

네트워크 통신을 7개의 독립된 계층으로 나눈 **참조 모델**이다.
각 계층은 하나의 역할만 담당하며, 트러블슈팅 시 문제를 계층별로 분리할 수 있다.

계층	이름	역할	프로토콜/장비 예시	DevOps 연관
7	Application	사용자 애플리케이션에 네트워크 서비스 제공	HTTP, HTTPS, FTP, SMTP, DNS, SSH	API 통신, 웹 서비스
6	Presentation	데이터 형식 변환, 암호화/복호화, 압축	SSL/TLS, JPEG, ASCII, JSON	TLS 인증서 관리
5	Session	세션 설정, 유지, 종료 관리	NetBIOS, RPC	연결 상태 관리
4	Transport	종단 간 데이터 전송, 신뢰성 보장	TCP, UDP	포트 설정, 방화벽 규칙
3	Network	논리적 주소 지정, 라우팅	IP, ICMP, ARP	VPC, 서브넷, 라우팅 테이블
2	Data Link	물리적 주소 지정, 프레임 전송, 오류 검출	Ethernet, MAC, PPP	네트워크 인터페이스
1	Physical	비트 단위 데이터를 물리 매체로 전송	케이블, 광섬유, Wi-Fi	데이터센터 물리 인프라

데이터 전송 흐름 (캡슐화/역캡슐화)

송신측(캡슐화: 위 → 아래)과 수신측(역캡슐화: 아래 → 위) 흐름:

계층	송신측 (캡슐화)	수신측 (역캡슐화)
7 Application	데이터 생성	데이터 수신
6 Presentation	암호화/인코딩	복호화/디코딩
5 Session	세션 관리	세션 관리
4 Transport	TCP/UDP 헤더 추가 → **세그먼트**	TCP/UDP 헤더 제거
3 Network	IP 헤더 추가 → **패킷**	IP 헤더 제거
2 Data Link	MAC 헤더 + 트레일러 추가 → **프레임**	MAC 헤더 제거
1 Physical	비트(0과 1) 전송	비트(0과 1) 수신

- 송신측은 7계층에서 1계층으로 내려가며 각 계층의 헤더를 추가(캡슐화)한다.
- 수신측은 1계층에서 7계층으로 올라가며 각 계층의 헤더를 제거(역캡슐화)한다.

계층별 데이터 단위 (PDU)

계층	PDU (Protocol Data Unit)
7~5	데이터 (Data)
4	세그먼트 (Segment, TCP) / 데이터그램 (Datagram, UDP)
3	패킷 (Packet)

계층	PDU (Protocol Data Unit)
2	프레임 (Frame)
1	비트 (Bit)

---

3. TCP/IP 모델

실제 인터넷에서 사용되는 **실무 모델**이다. OSI의 7계층을 4계층으로 단순화했다.

OSI vs TCP/IP 매핑

OSI 7계층	TCP/IP 4계층	주요 프로토콜
7. Application		
6. Presentation	**4. Application**	HTTP, DNS, SSH, FTP
5. Session		
4. Transport	**3. Transport**	TCP, UDP
3. Network	**2. Internet**	IP, ICMP, ARP
2. Data Link	**1. Network Interface**	Ethernet, Wi-Fi
1. Physical		

OSI vs TCP/IP 비교

항목	OSI 모델	TCP/IP 모델
계층 수	7개	4개
성격	이론적 참조 모델	실무 구현 모델
개발	ISO (국제표준화기구)	미 국방부 (DoD)
계층 경계	엄격하게 분리	유연하게 통합
용도	교육, 트러블슈팅 참조	실제 인터넷 프로토콜 스택
프로토콜 의존성	프로토콜 독립적	특정 프로토콜 기반

> **DevOps 실무에서**: 트러블슈팅 시 OSI 계층 개념으로 문제를 분리하고, 실제 통신은 TCP/IP 모델을 따른다. 둘 다 이해해야 한다.

---

4. IP 주소와 서브넷

IPv4 주소 구조

IPv4 주소는 4개의 옥텟(octet)으로 구성된 32비트 주소 체계이다.

구성 요소	설명
형식	`옥텟1.옥텟2.옥텟3.옥텟4` (예: `192.168.1.100`)
각 옥텟	8비트, 0~255 사이의 값
전체 길이	32비트
총 주소 수	약 43억 개 (2^{32})

공인 IP vs 사설 IP

구분	범위	용도
공인 IP (Public)	ISP가 할당한 고유 주소	인터넷에서 직접 접근 가능
사설 IP (Private)	10.0.0.0/8	대규모 네트워크 내부
사설 IP (Private)	172.16.0.0/12	중규모 네트워크 내부
사설 IP (Private)	192.168.0.0/16	소규모 네트워크 내부
루프백 (Loopback)	127.0.0.0/8 (주로 127.0.0.1)	자기 자신 (localhost)

> **사설 IP는 인터넷에서 직접 접근할 수 없다.** 외부 통신 시 NAT(Network Address Translation)을 통해 공인 IP로 변환된다.

서브넷과 CIDR

서브넷은 하나의 네트워크를 더 작은 네트워크로 나누는 것이다.
CIDR (Classless Inter-Domain Routing) 표기법으로 서브넷 범위를 표현한다.

CIDR 표기 구조 (`10.0.1.0/24` 예시):

구성 요소	설명
`10.0.1.0`	네트워크 주소
`/24`	서브넷 마스크 비트 수 (앞에서 24비트가 네트워크 부분)
서브넷 마스크	`255.255.255.0` (`11111111.11111111.11111111.00000000`)
네트워크 부분	앞 24비트 (`255.255.255`)
호스트 부분	뒤 8비트 (`.0`)

CIDR별 호스트 수

| CIDR | 서브넷 마스크 | 총 IP 수 | 사용 가능 호스트 수 | DevOps 용도 예시 | |:----😐:----------😐:---------😐:----------😐----------|| /32 | 255.255.255.255 | 1 | 1 (단일 호스트) | 특정 인스턴스 지정 | | /28 | 255.255.255.240 | 16 | 11 (AWS 예약 5개) | 소규모 서브넷 | | /24 | 255.255.255.0 | 256 | 251 | 일반적인 서브넷 | | /20 | 255.255.240.0 | 4,096 | 4,091 | 대규모 서브넷 | | /16 | 255.255.0.0 | 65,536 | 65,531 | VPC 전체 대역 | | /8 | 255.0.0.0 | 16,777,216 | 16,777,211 | 대기업 내부 네트워크 |

> **AWS VPC에서는** 각 서브넷의 처음 4개 + 마지막 1개 IP를 예약한다 (네트워크 주소, VPC 라우터, DNS 서버, 예비, 브로드캐스트). 그래서 /24 서브넷의 실제 사용 가능 호스트는 256 - 5 = 251개.

서브넷 설계 예시 (AWS VPC)

VPC 대역 `10.0.0.0/16` (65,536 IPs)을 용도 및 가용 영역(AZ)별로 분리한 구조:

서브넷 유형	AZ	CIDR	사용 가능 호스트	주요 리소스
퍼블릭 서브넷	AZ-a	10.0.1.0/24	251	웹 서버, NAT GW
퍼블릭 서브넷	AZ-b	10.0.2.0/24	251	웹 서버, NAT GW
프라이빗 서브넷	AZ-a	10.0.11.0/24	251	앱 서버
프라이빗 서브넷	AZ-b	10.0.12.0/24	251	앱 서버
DB 서브넷	AZ-a	10.0.21.0/24	251	RDS
DB 서브넷	AZ-b	10.0.22.0/24	251	RDS

IPv6

항목	IPv4	IPv6
주소 길이	32비트	128비트
주소 형식	10진수 (192.168.1.1)	16진수 (2001:0db8::1)
주소 수	약 43억	약 340간 (3.4 x 10^38)
NAT 필요	필요 (주소 부족)	불필요 (충분한 주소)
헤더 크기	가변 (20~60바이트)	고정 (40바이트)
보안	IPsec 선택	IPsec 내장

> AWS VPC에서는 IPv4와 IPv6를 동시에 사용하는 **듀얼 스택(Dual-Stack)** 구성을 지원한다.

---

5. TCP와 UDP

TCP (Transmission Control Protocol)

연결 지향적, **신뢰성 보장** 프로토콜. 데이터 순서 보장, 오류 검출, 재전송을 수행한다.

TCP 3-Way Handshake (연결 수립)

단계	방향	패킷	내용
1	클라이언트 → 서버	SYN (seq=100)	"연결하고 싶어요"
2	서버 → 클라이언트	SYN-ACK (seq=200, ack=101)	"좋아요, 저도요"
3	클라이언트 → 서버	ACK (seq=101, ack=201)	"확인, 시작합시다"

→ 3단계 완료 후 **연결 수립** 및 데이터 송수신 시작

TCP 4-Way Handshake (연결 종료)

단계	방향	패킷	내용
1	클라이언트 → 서버	FIN	"종료할게요"
2	서버 → 클라이언트	ACK	"알겠어요"
3	서버 → 클라이언트	FIN	"저도 종료할게요"
4	클라이언트 → 서버	ACK	"확인"

→ 4단계 완료 후 **연결 종료**

UDP (User Datagram Protocol)

비연결형, **신뢰성 비보장** 프로토콜. Handshake 없이 데이터를 바로 전송한다.

UDP 전송 특징:

특징	내용
전송 방식	클라이언트 → 서버로 단방향 데이터 전송
도착 확인	없음 (ACK 없음)
순서 보장	없음
재전송	없음

TCP vs UDP 비교

항목	TCP	UDP
연결 방식	연결 지향 (3-Way Handshake)	비연결형
신뢰성	보장 (ACK, 재전송)	비보장
순서 보장	보장	비보장
속도	상대적으로 느림 (오버헤드)	빠름 (오버헤드 최소)
헤더 크기	20~60 바이트	8 바이트
흐름/혼잡 제어	있음	없음
사용 사례	HTTP, SSH, FTP, SMTP, 데이터베이스	DNS 쿼리, 스트리밍, VoIP, 게임
DevOps 연관	웹 서비스, API, 파일 전송	DNS 해석, 로그 전송 (syslog), 모니터링

---

6. 포트와 소켓

포트 (Port)

포트는 하나의 IP 주소에서 **여러 서비스를 구분**하기 위한 번호이다.
범위: 0~65535

범위	이름	설명
0 ~ 1023	Well-Known Ports	표준 서비스에 할당 (관리자 권한 필요)
1024 ~ 49151	Registered Ports	애플리케이션이 등록하여 사용
49152 ~ 65535	Dynamic/Ephemeral Ports	클라이언트가 임시로 사용

DevOps에서 자주 쓰는 포트

포트	프로토콜	서비스	DevOps 용도
22	TCP	SSH	서버 원격 접속, Git (SSH)
53	TCP/UDP	DNS	도메인 이름 해석
80	TCP	HTTP	웹 서비스 (비암호화)
443	TCP	HTTPS	웹 서비스 (암호화)
3000	TCP	개발 서버	Grafana, React 개발 서버 등
3306	TCP	MySQL	데이터베이스 접속
5432	TCP	PostgreSQL	데이터베이스 접속
5672	TCP	AMQP	RabbitMQ 메시지 큐
6379	TCP	Redis	캐시/세션 스토어
8080	TCP	HTTP (대체)	Jenkins, 앱 서버 대체 포트
8443	TCP	HTTPS (대체)	Kubernetes API 서버 등
9090	TCP	Prometheus	모니터링 메트릭 수집
9200	TCP	Elasticsearch	로그 검색/분석
27017	TCP	MongoDB	NoSQL 데이터베이스

소켓 (Socket)

소켓은 **IP 주소 + 포트 번호**의 조합으로, 네트워크 통신의 끝점(Endpoint)이다.

- 형식: `IP 주소:포트 번호` (예: `192.168.1.10:443`)

소켓 통신 예시:

구분	소켓	설명
클라이언트 소켓	`192.168.1.50:52314`	임시(동적) 포트 사용
서버 소켓	`10.0.1.20:443`	HTTPS 서비스 포트

> **방화벽/보안 그룹 설정 시**: 포트 번호를 정확히 알아야 필요한 트래픽만 허용하고 나머지를 차단할 수 있다.

---

7. DNS (Domain Name System)

DNS란?

도메인 이름을 IP 주소로 변환하는 분산 데이터베이스 시스템이다.
사람이 기억하기 쉬운 이름(example.com)을 컴퓨터가 이해하는 IP(93.184.216.34)로 바꿔준다.

DNS 해석 과정

사용자가 `api.example.com`에 접속 요청 시 아래 순서로 처리된다:

단계	구성 요소	동작	결과
1	로컬 DNS 캐시 (브라우저/OS)	캐시 확인	캐시 히트 시 즉시 IP 반환, 미스 시 다음 단계
2	DNS Resolver (재귀적 리졸버, ISP 또는 8.8.8.8 등)	재귀 질의 시작	Root DNS 서버에 질의
3	Root DNS 서버 (`.`)	`.com`을 관리하는 TLD 서버 주소 반환	`.com` 네임서버 주소
4	TLD DNS 서버 (`.com`)	`example.com`을 관리하는 네임서버 주소 반환	`example.com` 네임서버 주소
5	Authoritative DNS 서버 (`example.com`)	`api.example.com`의 IP 주소 반환	`93.184.216.34`
6	DNS Resolver	결과를 TTL 동안 캐싱 후 클라이언트에 전달	클라이언트가 IP 수신
7	클라이언트	`93.184.216.34`로 TCP 연결 시작	실제 통신 시작

DNS 레코드 유형

레코드	이름	설명	예시
A	Address	도메인 → IPv4 주소	example.com → 93.184.216.34
AAAA	IPv6 Address	도메인 → IPv6 주소	example.com → 2606:2800:220:1::248
CNAME	Canonical Name	도메인 → 다른 도메인 (별칭)	www.example.com → example.com
MX	Mail Exchange	메일 서버 지정	example.com → mail.example.com
NS	Name Server	네임서버 지정	example.com → ns1.example.com
TXT	Text	임의 텍스트 (인증, SPF 등)	도메인 소유권 확인, SPF 레코드

레코드	이름	설명	예시
SRV	Service	서비스 위치 정보	포트/프로토콜/호스트 지정
PTR	Pointer	IP → 도메인 (역방향 조회)	93.184.216.34 → example.com
SOA	Start of Authority	영역(zone) 기본 정보	기본 NS, 관리자, 시리얼 번호

TTL (Time To Live)

DNS 응답이 캐시에 유지되는 시간(초 단위).

TTL 값	시간	적합한 경우
60	1분	빠른 변경이 필요한 경우 (장애 대응, 마이그레이션)
300	5분	일반적인 서비스
3600	1시간	안정적인 레코드
86400	24시간	거의 변경되지 않는 레코드

DevOps에서의 DNS 활용

활용	설명
서비스 디스커버리	Kubernetes 내부 DNS (CoreDNS)로 서비스 간 통신
로드 밸런서 연결	도메인을 ALB/NLB 엔드포인트에 CNAME/Alias로 매핑
SSL 인증서 발급	DNS 인증 (Route 53 + ACM)
Blue/Green 배포	DNS 전환으로 트래픽 라우팅 변경
장애 대응	TTL을 낮게 설정하여 빠르게 IP 전환
Geo-DNS	사용자 위치 기반으로 가장 가까운 서버로 라우팅

---

8. HTTP/HTTPS

HTTP 요청/응답 구조

HTTP 요청 (Request) 구성 요소:

구성 요소	예시	설명
요청 라인	`GET /api/users HTTP/1.1`	HTTP 메서드 + 경로 + 버전
Host 헤더	`Host: api.example.com`	대상 서버 도메인
Authorization 헤더	`Authorization: Bearer eyJhbGc...`	인증 토큰
Content-Type 헤더	`Content-Type: application/json`	요청 본문 형식
Accept 헤더	`Accept: application/json`	응답 형식 요구

구성 요소	예시	설명
요청 본문	(GET은 보통 없음)	POST/PUT 시 데이터 포함

HTTP 응답 (Response) 구성 요소:

구성 요소	예시	설명
상태 라인	`HTTP/1.1 200 OK`	HTTP 버전 + 상태 코드
Content-Type 헤더	`Content-Type: application/json`	응답 본문 형식
Content-Length 헤더	`Content-Length: 256`	응답 본문 크기
Cache-Control 헤더	`Cache-Control: max-age=3600`	캐시 제어
응답 본문	`{"users": [{"id": 1, "name": "Alice"}]}`	실제 응답 데이터

HTTP 메서드

메서드	용도	멱등성	안전성	예시
GET	리소스 조회	O	O	GET /api/users
POST	리소스 생성	X	X	POST /api/users
PUT	리소스 전체 수정	O	X	PUT /api/users/1
PATCH	리소스 부분 수정	X	X	PATCH /api/users/1
DELETE	리소스 삭제	O	X	DELETE /api/users/1
HEAD	헤더만 조회 (본문 없음)	O	O	HEAD /api/users
OPTIONS	허용 메서드 확인 (CORS)	O	O	OPTIONS /api/users

> **멱등성(Idempotent)**: 같은 요청을 여러 번 보내도 결과가 동일함.

HTTP 상태 코드

범위	분류	자주 쓰는 코드
1xx	정보	100 Continue
2xx	성공	200 OK, 201 Created, 204 No Content
3xx	리다이렉션	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	클라이언트 오류	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx	서버 오류	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

HTTPS = HTTP + TLS

구분	HTTP (평문)	HTTPS (암호화)
데이터 전송 방식	평문으로 전송	암호화하여 전송
전송 예시	`"password123"`	`"a8f3b2c1e9..."`
보안	누구나 볼 수 있음	복호화 키 없이 해독 불가

TLS Handshake 과정 (간략)

단계	방향	메시지	내용
1	클라이언트 → 서버	Client Hello	지원 TLS 버전 및 암호화 방식 목록 전송
2	서버 → 클라이언트	Server Hello + 인증서	선택된 암호화 방식과 서버 인증서 전송
3	클라이언트 → 서버	인증서 검증 + 키 교환	CA 확인, 대칭키 생성 및 전달
4	양방향	암호화 통신 시작	대칭키로 암호화된 데이터 송수신

HTTP/1.1 vs HTTP/2 vs HTTP/3

항목	HTTP/1.1	HTTP/2	HTTP/3
전송 프로토콜	TCP	TCP	QUIC (UDP 기반)
연결 방식	요청마다 연결 또는 Keep-Alive	단일 연결 멀티플렉싱	단일 연결 멀티플렉싱
헤더 압축	없음	HPACK	QPACK
서버 푸시	없음	지원	지원
Head-of-Line Blocking	TCP 레벨에서 발생	TCP 레벨에서 발생	해결됨 (UDP 기반)
연결 수립 시간	TCP + TLS (2~3 RTT)	TCP + TLS (2~3 RTT)	0~1 RTT

---

9. 라우팅과 게이트웨이

라우팅이란?

패킷이 출발지에서 목적지까지 도달하는 **경로를 결정**하는 과정이다.

라우팅 테이블

라우터/VPC가 패킷의 목적지를 보고 어떤 경로로 전달할지 결정하는 규칙 목록이다.

목적지 CIDR	다음 홉 (Next Hop)	설명
10.0.0.0/16	local	VPC 내부 통신
0.0.0.0/0	igw-12345	인터넷으로 나가는 트래픽
172.16.0.0/16	pcx-67890	VPC 피어링을 통한 통신

게이트웨이 유형 (AWS 기준)

게이트웨이	역할	방향
Internet Gateway (IGW)	VPC와 인터넷 연결	양방향 (인바운드 + 아웃바운드)
NAT Gateway	프라이빗 서브넷의 아웃바운드 인터넷 접속	아웃바운드만 (인바운드 차단)
Virtual Private Gateway (VGW)	VPN 연결을 위한 게이트웨이	온프레미스 연결
Transit Gateway (TGW)	여러 VPC와 온프레미스를 중앙 허브로 연결	다방향
VPC Endpoint	AWS 서비스에 프라이빗 연결	인터넷 경유 없이 AWS 서비스 접근

NAT (Network Address Translation)

프라이빗 IP를 가진 인스턴스가 인터넷과 통신할 수 있도록 IP 주소를 변환한다.

NAT 동작 흐름:

단계	구간	출발지 IP	목적지 IP	설명
요청 (→)	프라이빗 인스턴스 → NAT GW	10.0.2.50	8.8.8.8	원본 요청
요청 (→)	NAT GW → 인터넷	52.1.2.3 (공인 IP)	8.8.8.8	IP 변환 후 전송
응답 (←)	인터넷 → NAT GW	8.8.8.8	52.1.2.3	응답 수신
응답 (←)	NAT GW → 프라이빗 인스턴스	8.8.8.8	10.0.2.50	IP 역변환 후 전달

---

10. 로드 밸런싱

로드 밸런서란?

들어오는 트래픽을 **여러 서버에 분산**하여 가용성과 확장성을 보장한다.

로드 밸런서 동작 원리

사용자 요청이 로드 밸런서를 거쳐 정상 서버들에 분산된다.

구성 요소	상태	설명
사용자 요청	-	단일 진입점으로 유입
Load Balancer	-	트래픽을 하위 서버로 분배
서버 A	정상	트래픽 수신
서버 B	정상	트래픽 수신
서버 C	비정상	헬스 체크 실패 → 자동 제외

요청 흐름: 사용자 요청 → Load Balancer → 서버 A / 서버 B (서버 C는 비정상으로 제외)

AWS 로드 밸런서 유형

유형	약어	OSI 계층	프로토콜	적합한 경우
Application LB	ALB	L7	HTTP, HTTPS, gRPC	웹 앱, API, 경로 기반 라우팅
Network LB	NLB	L4	TCP, UDP, TLS	고성능, 초저지연, 고정 IP 필요 시
Gateway LB	GWLB	L3	IP	서드파티 보안 어플라이언스 앞단
Classic LB	CLB	L4/L7	TCP, HTTP	레거시 (신규 사용 비권장)

로드 밸런싱 알고리즘

알고리즘	설명
Round Robin	서버에 순서대로 분배
Least Connections	현재 연결 수가 가장 적은 서버에 분배
Weighted Round Robin	서버별 가중치에 따라 분배
IP Hash	클라이언트 IP 기반으로 고정 서버 할당
Least Response Time	응답 시간이 가장 빠른 서버에 분배

헬스 체크 (Health Check)

로드 밸런서가 주기적으로 서버 상태를 확인하여 비정상 서버를 자동 제외한다.

ALB 헬스 체크 설정 예시:

설정 항목	값	설명
프로토콜	HTTP	헬스 체크 프로토콜
경로	/health	헬스 체크 엔드포인트
포트	8080	헬스 체크 대상 포트
정상 임계값	3회 연속 200 OK	이 조건 충족 시 정상으로 판정
비정상 임계값	2회 연속 실패	이 조건 충족 시 비정상 → 트래픽 제외
간격	30초	헬스 체크 주기
시간 초과	5초	응답 대기 최대 시간

---

11. 방화벽과 접근 제어

보안 그룹 vs 네트워크 ACL (AWS 기준)

항목	보안 그룹 (Security Group)	네트워크 ACL (NACL)
적용 대상	인스턴스 (ENI) 레벨	서브넷 레벨

항목	보안 그룹 (Security Group)	네트워크 ACL (NACL)
상태 관리	Stateful	Stateless
규칙 유형	허용만	허용 + 거부
규칙 평가	모든 규칙 동시 평가	번호 순서대로 평가 (낮은 번호 우선)
기본 동작	인바운드: 전부 차단, 아웃바운드: 전부 허용	모든 트래픽 허용 (기본 NACL)
변경 반영	즉시	즉시

> **Stateful vs Stateless**
>
> - **Stateful** (보안 그룹): 인바운드 허용 시 응답 트래픽은 자동 허용. 아웃바운드 규칙 불필요.
> - **Stateless** (NACL): 인바운드/아웃바운드 각각 별도 규칙 필요. 응답 트래픽도 명시적으로 허용해야 함.

보안 그룹 설정 예시

웹 서버 보안 그룹 — 인바운드 규칙:

프로토콜	포트	소스	설명
TCP	80	0.0.0.0/0	HTTP (전체 허용)
TCP	443	0.0.0.0/0	HTTPS (전체 허용)
TCP	22	10.0.0.0/16	SSH (VPC 내부에서만)

웹 서버 보안 그룹 — 아웃바운드 규칙:

프로토콜	포트	대상	설명
All	All	0.0.0.0/0	모든 아웃바운드 허용

앱 서버 보안 그룹 — 인바운드 규칙:

프로토콜	포트	소스	설명
TCP	8080	sg-webserver	웹 서버 SG에서만 허용
TCP	22	sg-bastion	Bastion 호스트에서만 SSH

DB 보안 그룹 — 인바운드 규칙:

프로토콜	포트	소스	설명
TCP	5432	sg-appserver	앱 서버 SG에서만 PostgreSQL 허용

---

12. 클라우드 네트워킹 (VPC)

VPC 아키텍처 전체 구조

인터넷 → IGW → 퍼블릭 서브넷 → 프라이빗 서브넷 → DB 서브넷 순으로 계층화된 구조이다. 각 계층은 두 개의 가용 영역(AZ-a, AZ-b)에 걸쳐 이중화된다.

계층	AZ-a	AZ-b	주요 리소스
인터넷 진입점	IGW (Internet Gateway)	IGW (Internet Gateway)	인터넷 트래픽 진입/진출
퍼블릭 서브넷	10.0.1.0/24	10.0.2.0/24	ALB, NAT Gateway
프라이빗 서브넷	10.0.11.0/24	10.0.12.0/24	EC2 앱 서버, ECS Fargate
DB 서브넷	10.0.21.0/24 (RDS Primary)	10.0.22.0/24 (RDS Standby)	RDS (다중 AZ)

트래픽 흐름: 인터넷 → IGW → 퍼블릭 서브넷(ALB) → 프라이빗 서브넷(앱 서버) → DB 서브넷(RDS)

퍼블릭 서브넷 vs 프라이빗 서브넷

항목	퍼블릭 서브넷	프라이빗 서브넷
라우팅	IGW로 라우팅 (0.0.0.0/0 → igw)	NAT GW로 라우팅 (0.0.0.0/0 → nat)
공인 IP	자동 할당 가능	할당 없음
인터넷 접근	양방향 (인바운드 + 아웃바운드)	아웃바운드만 (NAT 경유)
배치 리소스	ALB, NAT Gateway, Bastion	앱 서버, DB, 내부 서비스

VPC 연결 옵션

연결 방식	설명	적합한 경우
VPC Peering	두 VPC 간 1:1 프라이빗 연결	소수 VPC 간 연결
Transit Gateway	중앙 허브를 통한 다대다 연결	다수 VPC + 온프레미스
VPN (Site-to-Site)	인터넷을 통한 암호화 터널	온프레미스 연결 (저비용)
Direct Connect	AWS까지 전용 물리 회선	고대역폭, 저지연 필요 시
VPC Endpoint	AWS 서비스에 프라이빗 연결	S3, DynamoDB 등 내부 접근
PrivateLink	서비스를 프라이빗하게 노출	SaaS 서비스 연결

---

13. 컨테이너 네트워킹

Docker 네트워크 모드

모드	설명	격리 수준	사용 사례
bridge (기본)	가상 브릿지(docker0)를 통한 연결	높음	단일 호스트 컨테이너 간 통신
host	호스트 네트워크 직접 사용	없음	네트워크 오버헤드 최소화 필요 시
none	네트워크 없음	최고	네트워크 불필요한 배치 작업

모드	설명	격리 수준	사용 사례
overlay	여러 호스트에 걸친 가상 네트워크	높음	Docker Swarm, 멀티호스트
macvlan	컨테이너에 물리 MAC 주소 할당	높음	레거시 앱, 물리 네트워크 직접 연결

Docker Bridge 네트워크

호스트 머신 내에서 컨테이너들이 `docker0` 브릿지를 통해 서로 통신하는 구조이다.

구성 요소:

구성 요소	주소/설명
호스트 외부 네트워크 인터페이스	eth0 (호스트 IP: 192.168.1.100)
docker0 브릿지	172.17.0.1 — 컨테이너들의 게이트웨이
컨테이너 A	172.17.0.2 — veth 인터페이스로 docker0에 연결
컨테이너 B	172.17.0.3 — veth 인터페이스로 docker0에 연결
컨테이너 C	172.17.0.4 — veth 인터페이스로 docker0에 연결

통신 방식:

- 컨테이너 간 통신: `docker0` 브릿지를 통해 직접 통신
- 외부 통신: NAT (iptables)를 통해 호스트 IP로 변환
- 포트 매핑: `-p 8080:80` 형식 (호스트:8080 → 컨테이너:80)

---

14. Kubernetes 네트워킹

Kubernetes 네트워킹 4가지 문제

Kubernetes는 다음 4가지 네트워킹 문제를 해결해야 한다.

문제	해결 방식
같은 Pod 내 컨테이너 간 통신	localhost 공유 (동일 네트워크 네임스페이스)
Pod 간 통신	플랫 네트워크 — 모든 Pod가 고유 IP로 직접 통신, NAT 없음
Service와 Pod 간 통신	Service 추상화 (ClusterIP, kube-proxy)
외부에서 클러스터 내부 접근	NodePort, LoadBalancer, Ingress

Pod 네트워킹 모델

노드별 Pod 구성 및 통신 방식:

구분	Node 1	Node 2
Pod A	10.244.1.2 (컨테이너 1: :8080, 컨테이너 2: :3000)	—

구분	Node 1	Node 2
Pod B	10.244.1.3	—
Pod C	—	10.244.2.2 (컨테이너 1: :8080)
Pod D	—	10.244.2.3
네트워크 플러그인	CNI (Calico, Cilium 등)	CNI (Calico, Cilium 등)

통신 규칙:

- Pod 내 컨테이너: `localhost`로 통신 (포트만 다르면 됨)
- 같은 노드의 Pod 간: CNI 플러그인이 가상 브릿지로 연결
- 다른 노드의 Pod 간: 오버레이 네트워크로 캡슐화하여 전달
- 모든 Pod는 NAT 없이 고유 IP로 직접 통신 가능

CNI (Container Network Interface)

Kubernetes는 네트워킹을 직접 구현하지 않고, **CNI 플러그인에 위임**한다.

CNI 플러그인	특징
Calico	L3 기반, BGP 라우팅, NetworkPolicy 지원, 가장 널리 사용
Cilium	eBPF 기반, 고성능, L7 정책, 관측성 우수
Flannel	간단한 오버레이 네트워크, 설정 쉬움, 기능 제한적
Weave Net	멀티캐스트 지원, 암호화 내장, 소규모 클러스터
AWS VPC CNI	Pod에 VPC IP 직접 할당, AWS 네이티브 통합

Kubernetes Service 유형

Service 유형	접근 범위	설명
ClusterIP	클러스터 내부	기본값. 내부 통신용 가상 IP 할당
NodePort	클러스터 외부	모든 노드의 특정 포트(30000~32767)로 접근
LoadBalancer	클러스터 외부	클라우드 로드 밸런서 자동 생성 (ALB/NLB 등)
ExternalName	클러스터 외부	외부 DNS를 CNAME으로 매핑

Service 동작 원리

외부 트래픽이 클러스터 내부 Pod로 전달되는 흐름:

단계	구성 요소	설명
1	외부 사용자	트래픽 발생

단계	구성 요소	설명
2	LoadBalancer (AWS ALB/NLB)	외부 트래픽 수신
3	NodePort: 30080	노드 포트로 전달
4	ClusterIP: 10.96.0.10:80	kube-proxy가 라우팅
5	Pod 분배	kube-proxy가 라운드 로빈으로 분배: Pod A (10.244.1.2:8080) / Pod B (10.244.1.3:8080) / Pod C (10.244.2.2:8080)

흐름 요약: 외부 사용자 → LoadBalancer → NodePort(30080) → ClusterIP(10.96.0.10:80) → Pod A / Pod B / Pod C

Ingress

L7(HTTP/HTTPS) 레벨에서 외부 트래픽을 클러스터 내부 서비스로 라우팅한다.

Ingress 라우팅 규칙 예시:

호스트 + 경로	대상 Service
`api.example.com/users`	User Service (ClusterIP)
`api.example.com/orders`	Order Service (ClusterIP)
`www.example.com`	Frontend Service (ClusterIP)

트래픽 흐름: 사용자 요청 → Ingress Controller (Nginx, Traefik, ALB Ingress 등) → 경로별 Service로 라우팅

```yaml
# Ingress 리소스 예시
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /users
            pathType: Prefix
            backend:
              service:
                name: user-service
                port:
                  number: 80
          - path: /orders
```

```
            pathType: Prefix
            backend:
              service:
                name: order-service
                port:
                  number: 80
```

Kubernetes 내부 DNS (CoreDNS)

Kubernetes는 **CoreDNS**를 통해 Service와 Pod에 대한 DNS 해석을 자동으로 제공한다.

DNS 형식	예시
같은 네임스페이스	`user-service`
다른 네임스페이스	`user-service.production.svc.cluster.local`
전체 FQDN	`<service>.<namespace>.svc.cluster.local`
Pod DNS	`10-244-1-2.default.pod.cluster.local`

---

15. 서비스 메시 (Service Mesh)

서비스 메시란?

마이크로서비스 간 통신을 관리하는 **전용 인프라 계층**이다.
애플리케이션 코드 변경 없이 트래픽 관리, 보안, 관측성을 제공한다.

서비스 메시가 필요한 이유

서비스 메시 없이:

문제	설명
직접 통신	서비스 A → 서비스 B → 서비스 C로 직접 호출
개별 구현 부담	각 서비스가 재시도, 타임아웃, 인증, 로깅을 직접 구현해야 함
확장성 문제	서비스 수가 늘어나면 관리 불가능

서비스 메시 사용 시:

구성 요소	역할
서비스 A + Sidecar Proxy (P)	트래픽이 프록시를 통해 전달됨
서비스 B + Sidecar Proxy (P)	트래픽이 프록시를 통해 전달됨
서비스 C + Sidecar Proxy (P)	트래픽이 프록시를 통해 전달됨
컨트롤 플레인	설정, 인증서, 정책을 중앙 관리하여 각 프록시에 배포

- `P` = Sidecar Proxy (Envoy)

- 트래픽 관리, mTLS, 메트릭, 트레이싱을 프록시가 담당
- 애플리케이션 코드는 비즈니스 로직에만 집중

서비스 메시 아키텍처

구성 요소	역할
Data Plane (데이터 플레인)	각 Pod에 배치된 사이드카 프록시(Envoy)가 모든 트래픽을 가로채서 처리
Control Plane (컨트롤 플레인)	프록시에 설정(라우팅 규칙, 인증서, 정책)을 배포하고 관리

서비스 메시가 제공하는 기능

기능	설명
mTLS (상호 TLS)	서비스 간 통신을 자동으로 암호화. 양쪽 모두 인증서 검증
트래픽 라우팅	가중치 기반 라우팅, 카나리 배포, A/B 테스트
재시도/타임아웃	자동 재시도, 서킷 브레이커, 타임아웃 설정
관측성 (Observability)	메트릭, 분산 트레이싱, 액세스 로그 자동 수집
접근 제어	서비스 간 통신 정책 (어떤 서비스가 어떤 서비스에 접근 가능한지)
속도 제한 (Rate Limiting)	서비스별 요청 속도 제한

Istio vs Linkerd

| 항목 | Istio | Linkerd | |------|-------|---------|| | 프록시 | Envoy (C++) | Linkerd2-proxy (Rust) | | 기능 범위 | 매우 넓음 (L4~L7) | 핵심 기능 중심 | | 리소스 사용량 | 상대적으로 높음 | 경량 | | 학습 곡선 | 높음 | 낮음 | | 커뮤니티 | CNCF Graduated, 대규모 | CNCF Graduated, 중규모 | | 적합한 경우 | 고급 트래픽 관리, 대규모 조직 | 간단한 메시, 빠른 도입 |

> **Istio Ambient Mode (2024~)**: 사이드카 없이 노드 레벨의 ztunnel로 L4 mTLS를 처리하고, 필요한 서비스만 L7 waypoint 프록시를 사용하는 새로운 아키텍처. 리소스 사용량이 크게 줄어든다.

---

16. 네트워크 트러블슈팅

계층별 트러블슈팅 접근법

네트워크 문제는 **아래 계층부터 위로** 올라가며 확인한다.

문제 발생: "서비스에 접속이 안 됩니다"

단계	계층	확인 내용	명령어	결과 해석
1	L3	네트워크 도달성 확인	`ping 10.0.1.20`	응답 있음 → Step 2 / 응답 없음 → 라우팅/방화벽/보안 그룹 확인

단계	계층	확인 내용	명령어	결과 해석
2	DNS	DNS 해석 확인	`dig api.example.com`	IP 반환 → Step 3 / NXDOMAIN → DNS 레코드 확인
3	L4	포트 연결 확인	`nc -zv 10.0.1.20 8080`	Connection succeeded → Step 4 / Connection refused → 서비스 미실행 / Connection timed out → 방화벽/SG 차단
4	L7	HTTP 응답 확인	`curl -I https://api.example.com/health`	200 OK → 서비스 정상 / 5xx → 애플리케이션 로그 확인 / 연결 실패 → TLS 인증서 확인

주요 트러블슈팅 명령어

명령어	계층	용도	예시
`ping`	L3	호스트 도달성 확인 (ICMP)	`ping 10.0.1.20`
`traceroute`	L3	패킷 경로 추적 (각 홉 지연)	`traceroute api.example.com`
`mtr`	L3	ping + traceroute 통합 (실시간)	`mtr api.example.com`
`dig`	DNS	DNS 레코드 조회 (상세)	`dig api.example.com A`
`nslookup`	DNS	DNS 레코드 조회 (간단)	`nslookup api.example.com`
`nc` (netcat)	L4	TCP/UDP 포트 연결 테스트	`nc -zv 10.0.1.20 8080`
`ss`	L4	소켓 상태 확인 (netstat 대체)	`ss -tulpn`
`curl`	L7	HTTP 요청 테스트	`curl -I https://api.example.com`
`ip addr`	L2/L3	네트워크 인터페이스/IP 확인	`ip addr show`
`ip route`	L3	라우팅 테이블 확인	`ip route show`
`tcpdump`	All	패킷 캡처 (실시간 분석)	`tcpdump -i eth0 port 80`
`iptables -L`	L3/L4	방화벽 규칙 확인	`iptables -L -n`

자주 발생하는 네트워크 문제와 원인

증상	가능한 원인	확인 방법
Connection timed out	보안 그룹/NACL 차단, 라우팅 오류	`nc -zv`, 보안 그룹 규칙 확인
Connection refused	서비스 미실행, 포트 미리슨	`ss -tulpn`으로 리스닝 포트 확인
Name resolution failed	DNS 설정 오류, 레코드 없음	`dig`, `nslookup`
502 Bad Gateway	백엔드 서비스 다운, LB 헬스체크 실패	LB 타겟 그룹 헬스 상태 확인
503 Service Unavailable	서비스 과부하, 모든 인스턴스 비정상	인스턴스 상태, 오토스케일링 확인

증상	가능한 원인	확인 방법
504 Gateway Timeout	백엔드 응답 지연, 타임아웃 초과	LB 타임아웃 설정, 앱 성능 확인
TLS Handshake 실패	인증서 만료, 도메인 불일치, TLS 버전	`curl -v`, 인증서 유효기간 확인
간헐적 연결 실패	DNS 캐시 문제, 포트 고갈, 커넥션 풀 부족	`ss -s`, 커넥션 수 모니터링

17. 네트워크 보안 기초

Zero Trust 네트워크

"절대 신뢰하지 말고, 항상 검증하라 (Never Trust, Always Verify)"

기존 모델	Zero Trust 모델
내부 네트워크는 신뢰	내부/외부 구분 없이 모두 검증
경계(방화벽) 중심 보안	리소스 단위 접근 제어
VPN으로 내부 접근 허용	요청마다 인증/인가 수행
한 번 인증하면 전체 접근	최소 권한, 세션 단위 인증

서비스 간 암호화

방식	설명	적용 방법
TLS (한방향)	클라이언트가 서버 인증서를 검증	HTTPS 적용
mTLS (상호)	클라이언트와 서버가 서로 인증서를 검증	서비스 메시 (Istio, Linkerd)

NetworkPolicy (Kubernetes)

Pod 간 트래픽을 제어하는 Kubernetes 네이티브 방화벽.

```yaml
# 예시: backend 네임스페이스의 Pod는
#       frontend 네임스페이스에서 온 트래픽만 수신 허용
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-only
  namespace: backend
spec:
  podSelector: {}        # backend 네임스페이스의 모든 Pod에 적용
  policyTypes:
    - Ingress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              name: frontend
      ports:
```

```
        - protocol: TCP
          port: 8080
```

VPN (Virtual Private Network)

인터넷을 통해 **암호화된 터널**을 만들어 프라이빗 네트워크처럼 사용한다.

| 유형 | 설명 | AWS 서비스 | |------|------|----------|| | Site-to-Site VPN | 온프레미스 네트워크와 VPC를 연결 | AWS Site-to-Site VPN | | Client VPN | 개별 사용자가 VPC에 접속 | AWS Client VPN | | Direct Connect | 전용 물리 회선 (인터넷 미경유) | AWS Direct Connect |

---

18. 참고 자료

리소스	URL
DEV Community — Networking Basics for DevOps	https://dev.to/pallavivemula/--5bl
AWS 네트워킹 기초 가이드	https://aws.amazon.com/getting-started/aws-networking-essentials/
Kubernetes 네트워킹 심화	https://iamachs.com/blog/kubernetes-networking/part-1-demystifying-kubernetes-networking/
Istio 공식 문서 — Service Mesh	https://istio.io/latest/about/service-mesh/
Apptio — Istio & Envoy 가이드	https://www.apptio.com/topics/kubernetes/devops-tools/istio-envoy/
OSI vs TCP/IP 모델 비교	https://www.checkpoint.com/cyber-hub/network-security/what-is-the-osi-model-understanding-the-7-layers/osi-model-vs-tcp-ip-model/
GeeksforGeeks — OSI and TCP/IP Model	https://www.geeksforgeeks.org/computer-networks/difference-between-osi-model-and-tcp-ip-model/
10 Core Networking Concepts for DevOps	https://www.linkedin.com/pulse/10-core-networking-concepts-every-modern-devops-team-must-understand-dcrfc
Linux 네트워크 트러블슈팅 도구	https://codilime.com/blog/linux-network-troubleshooting/
18 Cloud Networking Concepts for DevOps	https://www.devopstraininginstitute.com/blog/18-cloud-networking-concepts-devops-engineers-must-know

---

> 이 문서는 2026년 4월 기준으로 작성되었습니다. 네트워크 기술과 클라우드 네트워킹은 계속 진화하므로, 공식 문서를 함께 참고하세요.